Introduction to
Computer Organisation and Architecture
UCCD1133

Chapter 3

Basic Concept of Logic

Disclaimer

- This slide may contain copyrighted material of which has not been specifically authorized by the copyright owner. The use of copyrighted materials are solely for educational purpose. If you wish to use this copyrighted material for other purposes, you must first obtain permission from the original copyright owner.

Outline

- Combinational circuit building blocks
  - Decoder
  - Multiplexer
  - Adder
  - Subtractor

Combinational Circuits Building Blocks

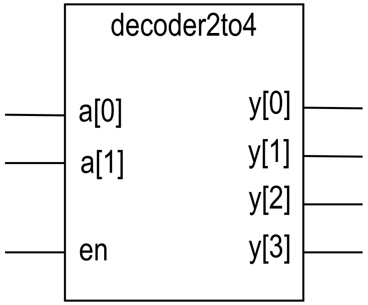
- Example of non-arithmetic building blocks:
  i. Decoder
  ii. Encoder and priority encoder
  iii. Demultiplexer
  iv. Multiplexer

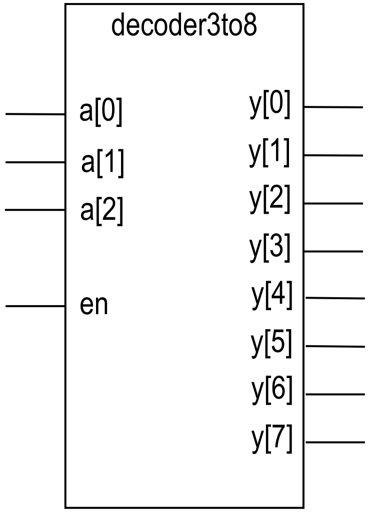
- Example of arithmetic building blocks:
  i. Adder
  ii. Subtractor
  iii. ALU (Arithmetic and Logic)
  iv. Multiplier
  v. Diviser
  vi. Comparator
  vii. Shifter

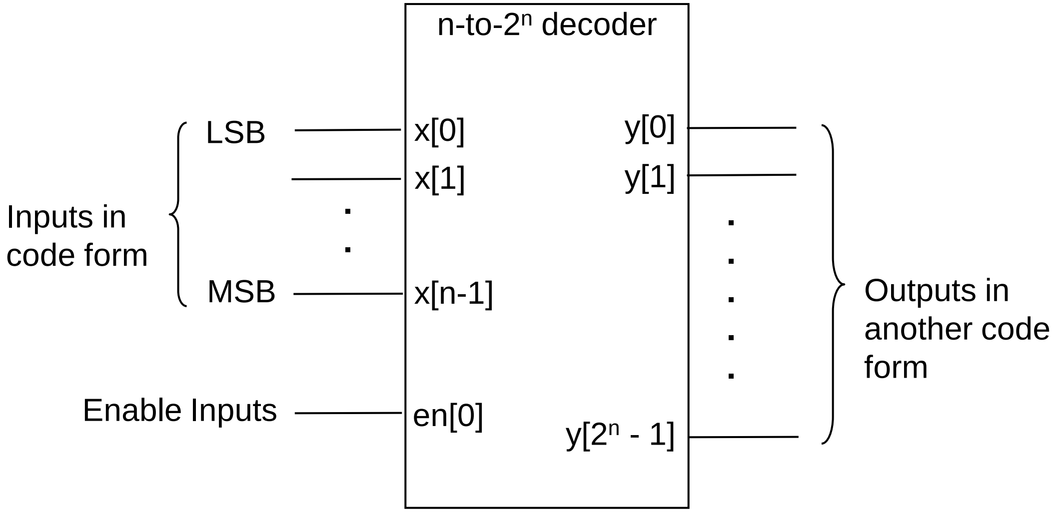
- What is standard circuits?
  i. Pre-designed circuits with at least correct functionalities
  ii. They speed up design work (modelling based on schematic, verification and implementation)
  iii. Collected as part of library

Decoder

- A decoder:
    i. Converts one type of code, x[n-1:0], into another type, y[2^n-1:0].
    ii. The enable inputs, en - to enable/ disable the outputs



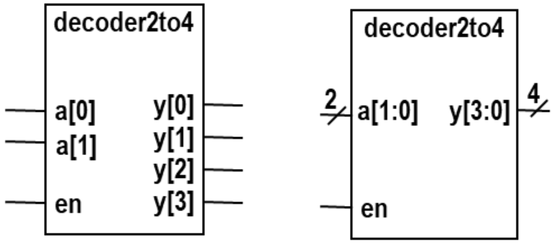




- A binary decoder
    i. Most basic type of decoder
    ii. Also known as n-to-2^n decoder – e.g. 3-to-8 decoder.
        ✓ Has n inputs and 2^n outputs.
    iii. Only one output (1-out-of-2^n) can be asserted
        ✓ To indicate/ identify the code that has been placed at input.
- Other types of decoder: display decoder

Decoder (2 to 4 Binary Decoder)

- Block diagram: 2 inputs and 4 outputs, plus one enable input.



- **Functional Behaviour** -(the in-out relationship) for binary decoder:
- only an output (1-out-of-2^n) can be asserted when a code is applied to the inputs.

Inputs			Outputs			
en	a[1]	a[0]	y[3]	y[2]	y[1]	y[0]
0	d	d	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

- **Logic Function**

Function:

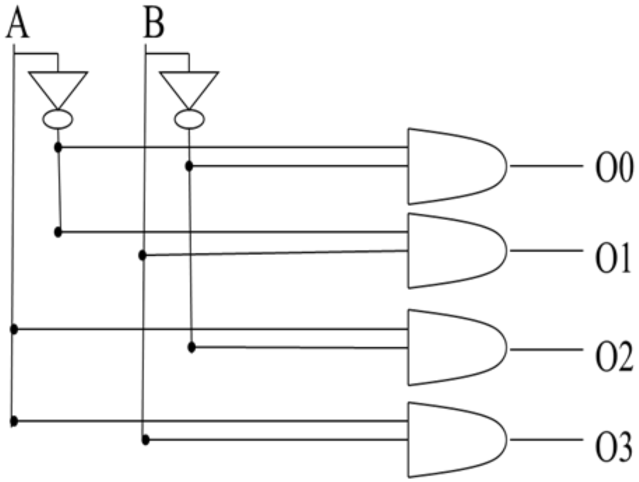
$O_3O_2O_1O_0$ = 0001 when A_1A_0 = 00 → $O0 = \overline{A1} \cdot \overline{A0}$

$O_3O_2O_1O_0$ = 0010 when A_1A_0 = 01 → $O1 = \overline{A1} \cdot A0$

$O_3O_2O_1O_0$ = 0100 when A_1A_0 = 10 → $O2 = A1 \cdot \overline{A0}$

$O_3O_2O_1O_0$ = 1000 when A_1A_0 = 11 → $O3 = A1 \cdot A0$

- Implementation using basic logic gates



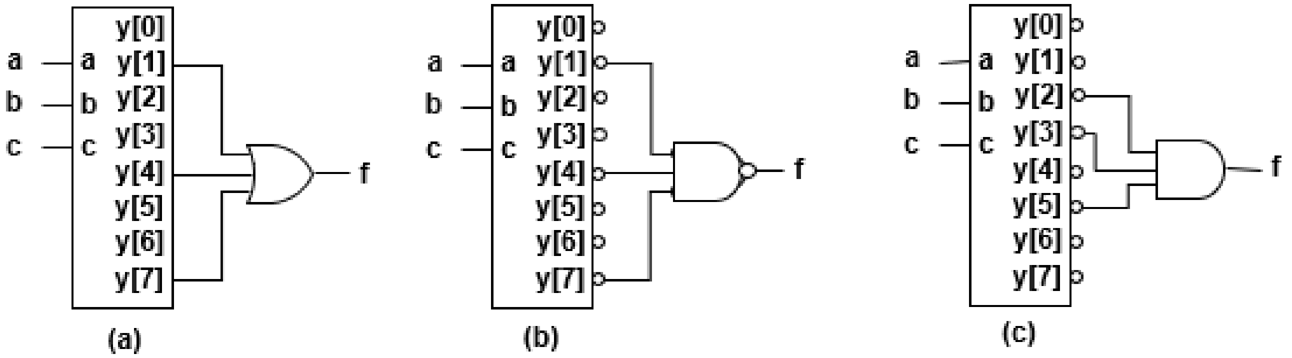
Decoder Application 1: Minterm Generation

- Notice the outputs of binary decoder represent all possible minterms.
  i. Can be used as a minterm generator.

Example10 : Implement the following functions.

1. f(A,B,C) = m1 + m4 +m7: use a decoder (with active-high outputs) with an OR gate.
2. f(A,B,C) = (m1’ m4’ m7’)’: use a decoder (with active-low outputs) with a NAND gate.
3. f(A,B,C) = m2’ m3’ m5’: use a decoder (with active-low outputs) with an AND gate.

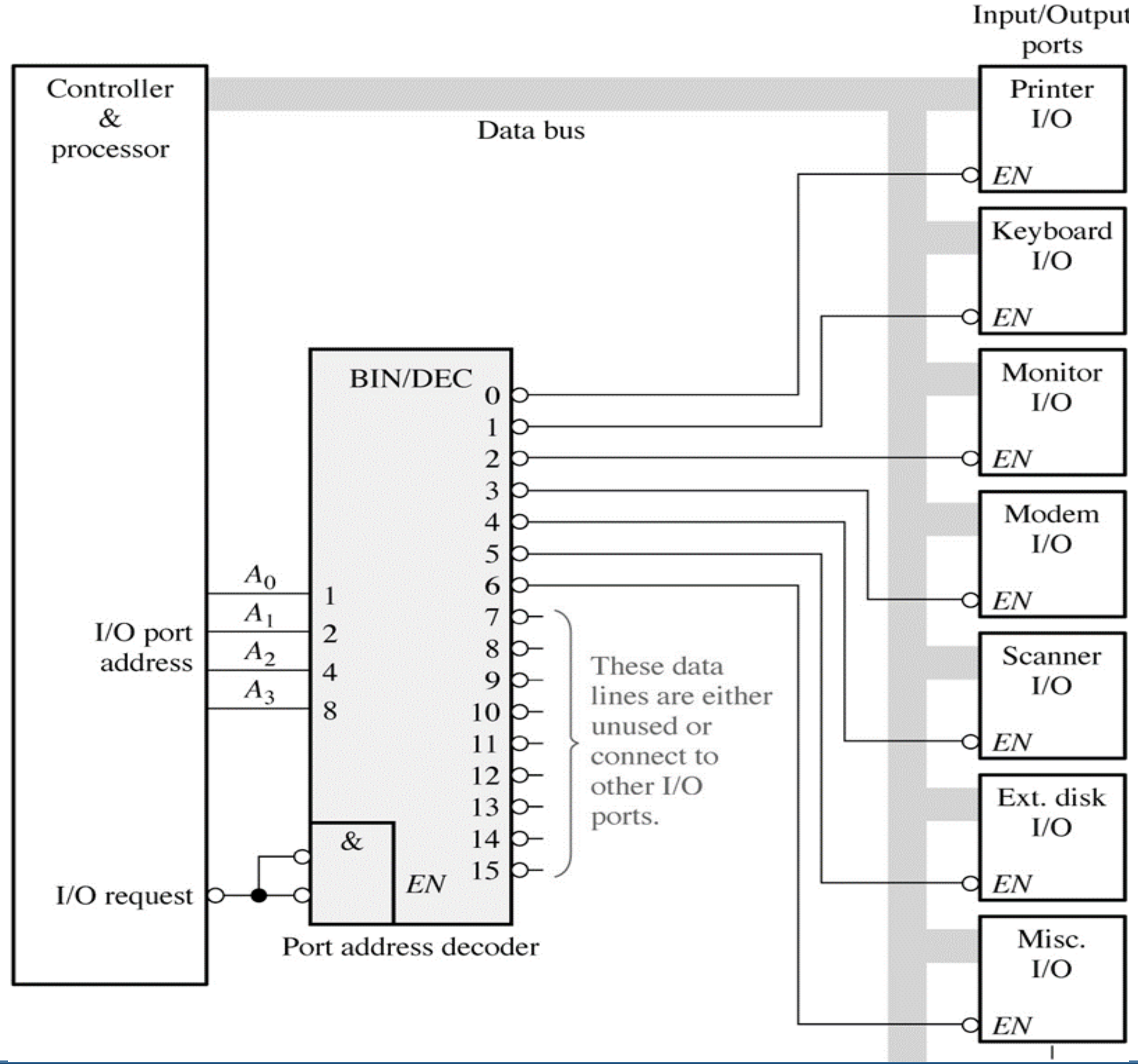
Solution:



Decoder Application 2: Device Enable/Control

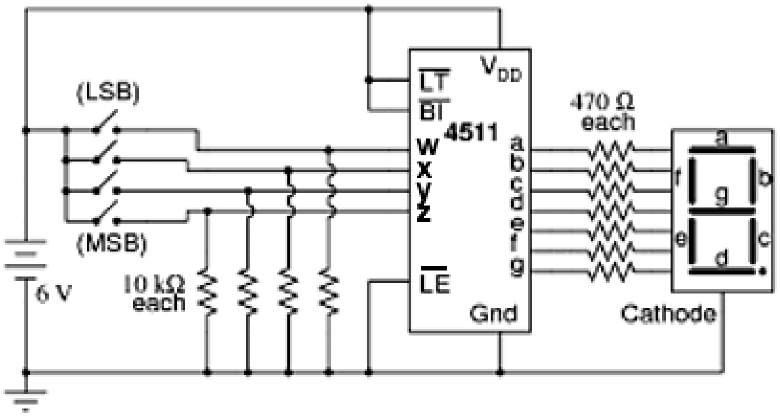
Example 11:

A 4-to-16 binary decoder to selectively enable one out of 7 I/O devices. 9 outputs are not used. Only one device is allowed to be active at any time.

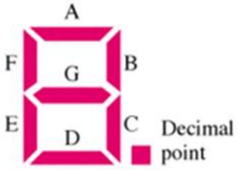


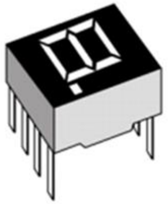
Display decoder

- A display decoder converts a code into a suitable format to drive a numeric display (LCD, 7-segment display, dot matrix display, etc)
- Example
  i. 7-segment display decoders (BCD-to-7 segment decoders)
  ii. Available in the market as standard components 7447 and 74X49.



Inputs	Outputs						
w x y z	a	b	c	d	e	f	g
0 0 0 0	1	1	1	1	1	1	0
0 0 0 1	0	1	1	0	0	0	0
0 0 1 0	1	1	0	1	1	0	1
0 0 1 1	1	1	1	1	0	0	1
0 1 0 0	0	1	1	0	0	1	1
0 1 0 1	1	0	1	1	0	1	1
0 1 1 0	1	0	1	1	1	1	1
0 1 1 1	1	1	1	0	0	1	0
1 0 0 0	1	1	1	1	1	1	1
1 0 0 1	1	1	1	1	0	1	1
1 0 1 0	d	d	d	d	d	d	d
1 0 1 1	d	d	d	d	d	d	d
1 1 0 0	d	d	d	d	d	d	d
1 1 0 1	d	d	d	d	d	d	d
1 1 1 0	d	d	d	d	d	d	d
1 1 1 1	d	d	d	d	d	d	d

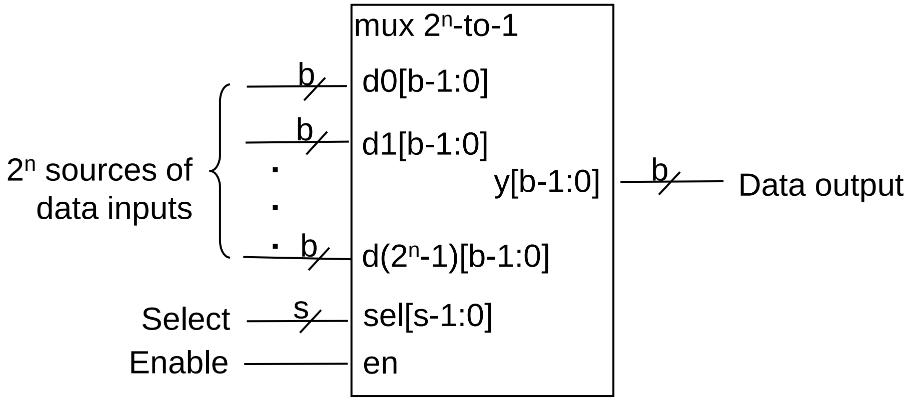




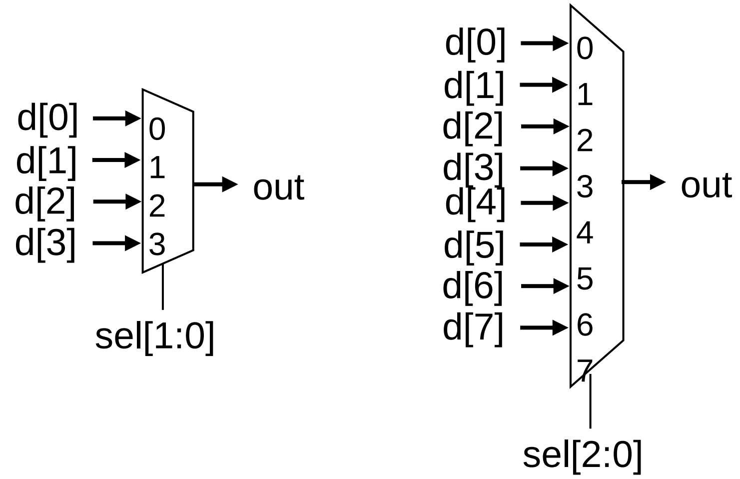
Multiplexer (Data Selector)

- A multiplexer (mux) is a circuit that has:
  i. 2^n data inputs
  ii. s data selectors (control inputs)
  iii. An output

- Like many other circuits, the mux can have additional enable input signal, en
  i. To enable or disable the operation of the mux
  ii. Set the output to z



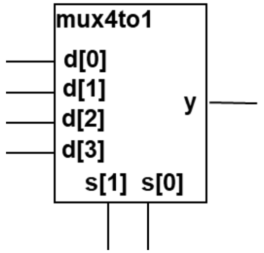
Block diagram of 2^n-to-1 mux



Graphical symbol of 4-to-1 mux and 8-to-1 mux

Multiplexer (4 to1 Multiplexer)

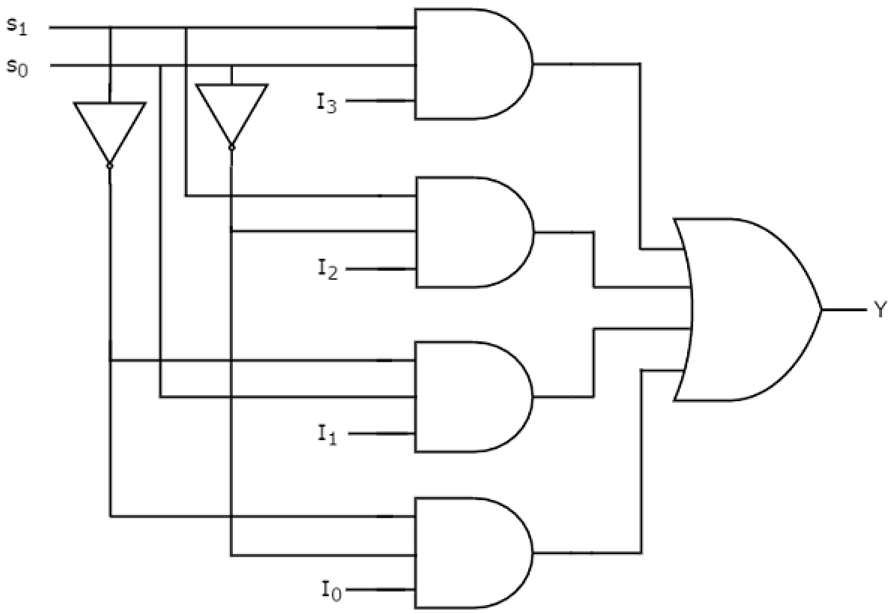
- Block diagram
  - A basic 4-to-1 multiplexer has the following signals.
  - The output signal y.
  - The input signals, d[3:0].
  - Since the multiplexer has 4 data inputs, it required 2 bits data-select ($4 = 2^2$).



- Functional Behaviour

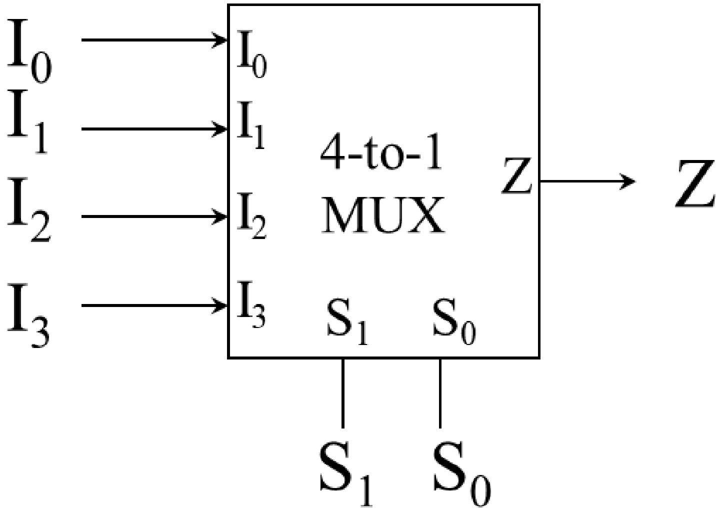
s[1]	s[0]	y
0	0	d[0]
0	1	d[1]
1	0	d[2]
1	1	d[3]

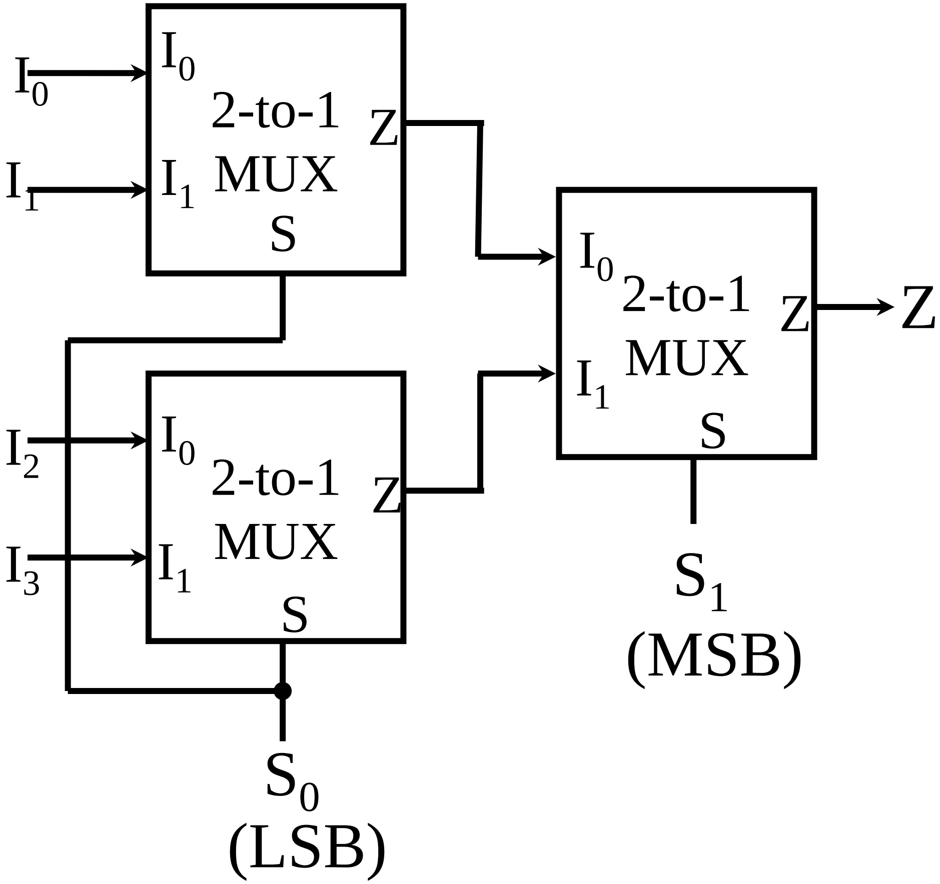
- Logic function
  ✓The logic function of the multiplexer:
  y = s[1]'s[0]'d[0] + s[1]'s[0] d[1] + s[1]s[0]'d[2] + s[1]s[0]d[3]



Multiplexer Expansion

- A few multiplexers can be combined to built a bigger multiplexer.
- For example: a 4-to-1 MUX can be built by combining three units of 2-to-1 MUX are required.





Application: Minterm Generation

- Select inputs -> generates minterms, while data lines are used to enable the minterms

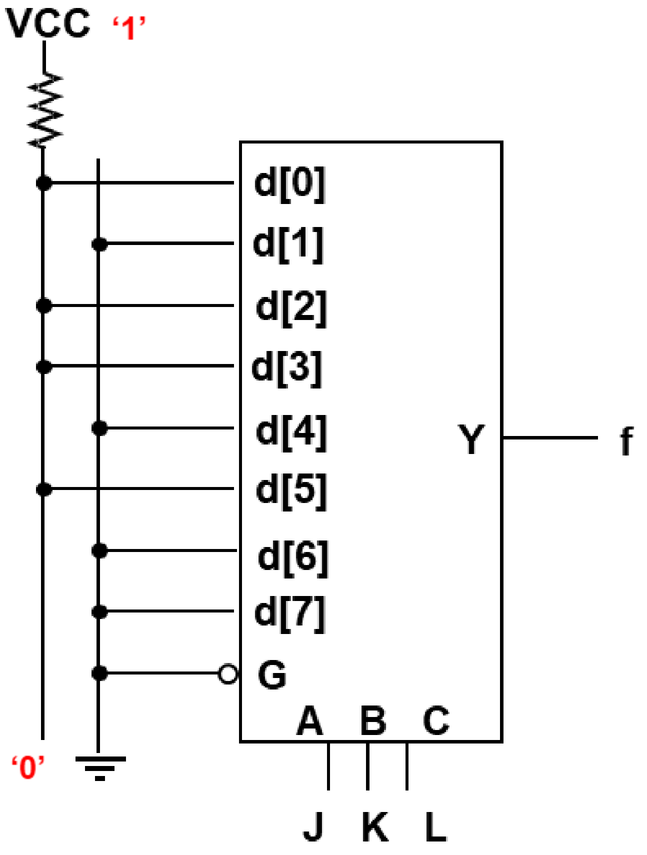
Example 12: Use a 8-to-1 multiplexer to realize $f(J,K,L) = \Sigma m(0, 2, 3, 5)$.

Solution

i. List the truth table for the function.

ii. Then map the content of the truth table to the mux.

J K L	f
0 0 0	1
0 0 1	0
0 1 0	1
0 1 1	1
1 0 0	0
1 0 1	1
1 1 0	0
1 1 1	0



Application: Minterm Generation

Example 13:

i. Implement f(a,b,c) = ab + b'c using 4-to-1 mux if a and b are used as the select signal of the mux.

Solution

Part (a)

- Express the logic function in canonical form.

  f = ab + b'c

  = ab(c' + c) + b'c(a' + a)

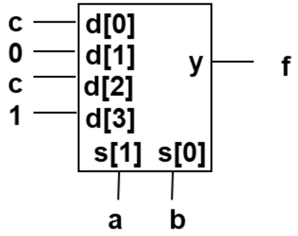
  = abc' + abc + a'b'c + ab'c

- Construct its truth table.

- Derive the reduced inputs (Mux inputs)

a	b	c	f	Mux inputs
0	0	0	0	d[0] = c
0	0	1	1	
0	1	0	0	d[1] = 0
0	1	1	0	
1	0	0	0	d[2] = c
1	0	1	1	
1	1	0	1	d[3] = 1
1	1	1	1	

- The final result:



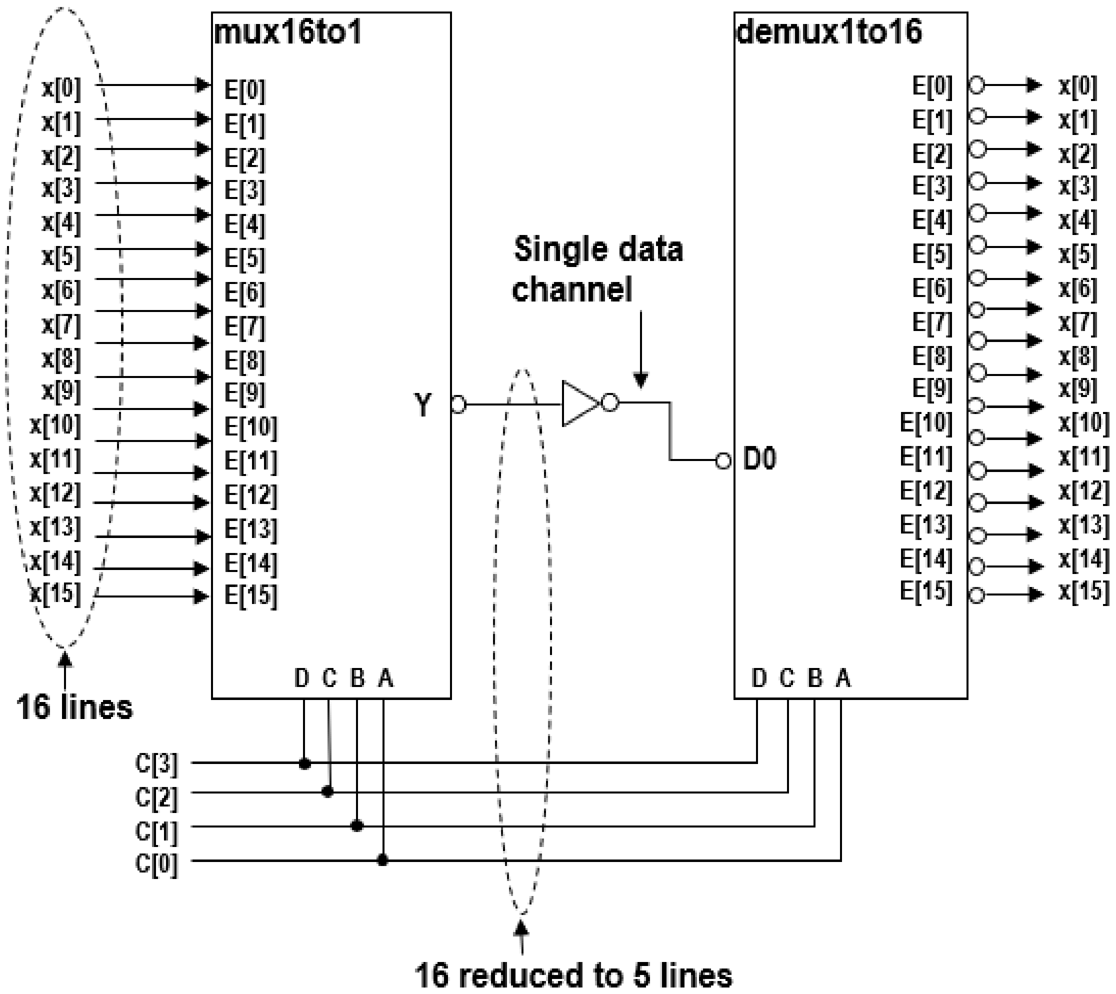
**If not specified, other variables can be used as the select input

Application of Mux/Demux: Data Routing/Transmission*

- Data must be switched from multiple sources to a destination.

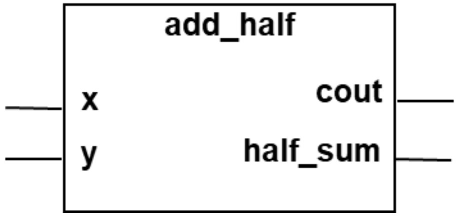
Example 14: Design a 16-line to 16-line mux/demux system.

- The purpose of the system is to reduce the number of wires used for transmission



Half Adder & Full Adder

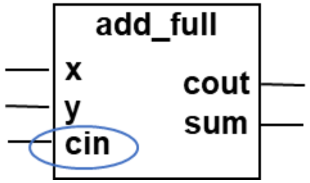
- A half-adder is a logic circuit that performs a single bit addition. It adds two bits, x and y, and produces two outputs, a sum bit and a carry out bit.
- There is no carry-in input
  i. carry-in is treated as 0.
- Block diagram



- Truth table:

x	y	cout	half_sum
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

- The main difference between a full adder and half-adder is that a full adder accepts the carry in C_{in}.
- Block diagram



- Truth table

cin	x	y	cout	sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Adder

- Addition is one of the most common operation in digital systems.
- Adder is an arithmetic circuit that adds two n-bit binary numbers.
- Can be implemented in combinational or sequential logic.
- Terminology for addition operation:

$$\underbrace{11110_2}_{\textbf{Augend}} + \underbrace{01100_2}_{\textbf{Addend}} = \underbrace{01010_2}_{\textbf{Sum}}$$

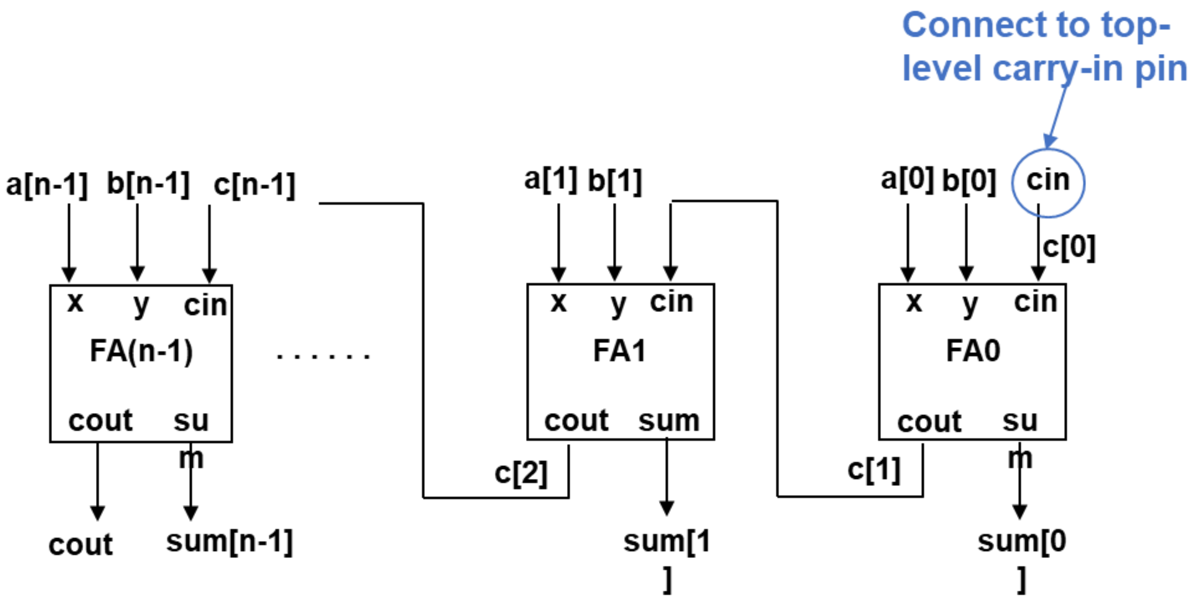
- The carry bit 1 is called the carry output, Cout of the third column and the carry in, Cin to the fourth column of addition.

	1←	1←	1←	0←	0←	
	1	1	1	1	0	Augend
						Carries
+	0	1	1	0	0	Addend
	0	1	0	1	0	Sum

N-bit adder: Carry –Ripple adder

- n-bit carry-ripple adder adds 2 n-bit binary numbers (signed and unsigned)
  - ✓ Result is (n+1)-bit: n-bit sum and a carry-out.
  - ✓ Each bit position requires a full-adder

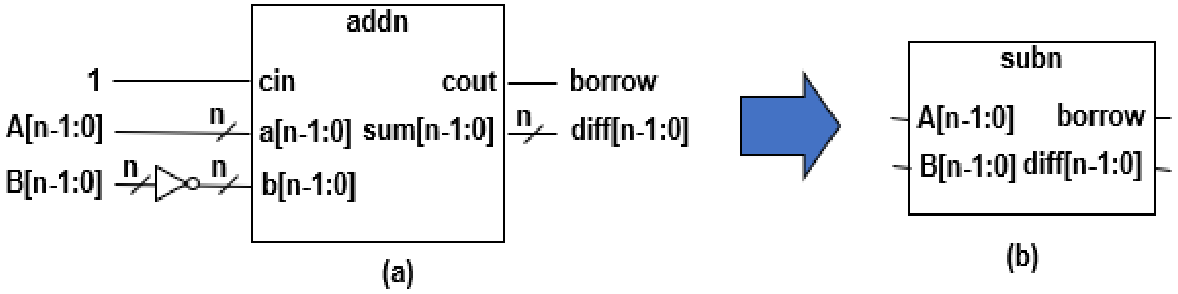
- Principle in constructing an n-bit carry-ripple adder
  - ✓ Cascading n full-adders (FA) through a chain of carry signals.
  - ✓ FA(i).cout => FA(i+1).cin where i = 0, 1, …. n-1



Subtractor Constructed from Adder

- Subtraction can be carried out using the adder circuit
- Based on the 2's complement arithmetic

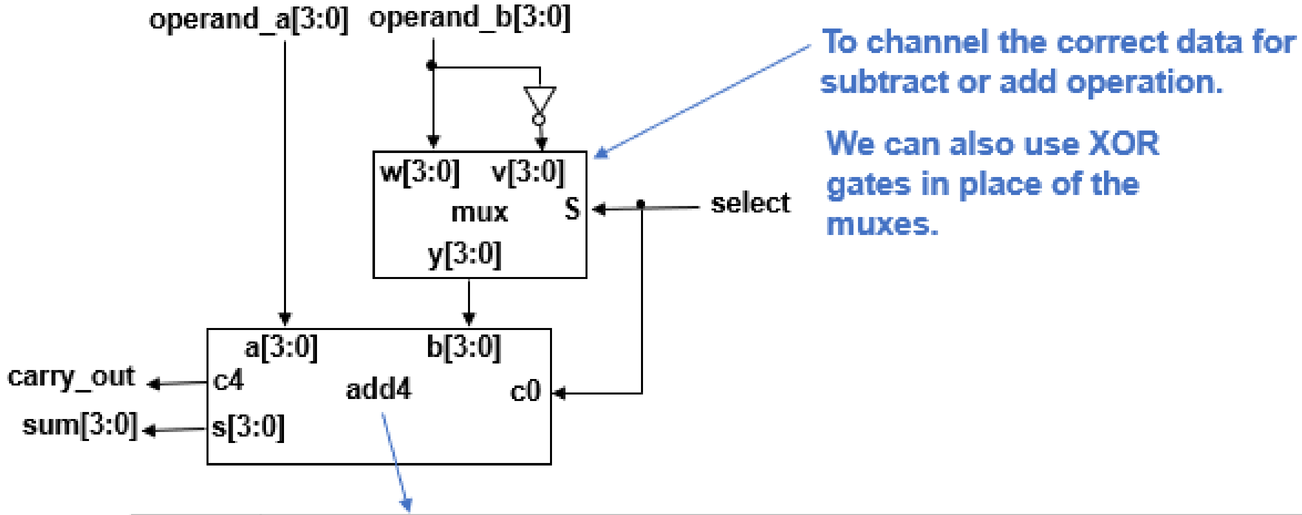
$$
\begin{aligned}
D_2 &= A_2 - B_2 \\
&= A_2 + [B]_2 \\
&= A_2 + (B_2)' + 1
\end{aligned}
$$



- However, the circuit can only perform one function i.e., subtraction.
  - Cannot perform addition.
- To perform both, add and subtract functions, combine the adder and subtracter circuits into one.

Subtraction/Addition Implementation Based on 2's Complement Arithmetic

- Construction of binary adder used to perform both addition and subtraction.



Signals	Addition Operation	Subtraction Operation
select	select = 0 Mux output, y[3:0] = w[3:0] = operand_b[3:0]	select = 1 Mux output, y[3:0] = v[3:0] = operand_b[3:0]'
c0	c0 = select = 0	c0 = select = 1
sum[3:0]	sum[3:0] = a[3:0] + b[3:0] + c0 = operand_a[3:0] + y[3:0] + 0 = operand_a[3:0] + w[3:0] = operand_a[3:0] + operand_b[3:0]	sum[3:0] = a[3:0] + b[3:0] + c0 = operand_a[3:0] + y[3:0] + 1 = operand_a[3:0] + v[3:0] + 1 = operand_a[3:0] + operand_b[3:0]' + 1

Overflow

- Two's complement numbers (2cns) uses complement arithmetic.
  - ✓ E.g. (A – B) can be performed by computing A + (-B) = A + B_{2C}
  - ✓ Only need a binary adder and complementing circuits to handle both add and subtract.
  - ✓ Easy hardware implementation.
- One problem of digital systems is that it have limited bits for data representation (limited range of signed values).
  - ✓ The decimal range for an n-bit digital system using 2cns is $(-2^{n-1}) \leq N \leq (2^{n-1} - 1)$.
  - ✓ E.g. an 8-bit digital system can only operate on values between -128 to 127.
- **If the result is out of the range, signed overflow occurs.**
  - ✓ The **out-of-range result cannot be used.**
  - ✓ Should design digital system that generates a warning signal. So that invalid numbers are not mistaken for correct results.

Overflow: Signed and Unsigned Numbers

- The computation for signed and unsigned overflows are not the same.

  Example

  Compute (– 9 – 5).

  $$-9 - 5 = (-9) + (-5)$$
  $$= [01001] + [00101]$$
  $$= 10111_{2c} + 11011_{2c}$$

Signed addition: input and output numbers treated as signed

Unsigned addition: input and output numbers treated as unsigned

	Two's complement	Signed	Unsigned
	10111_{2c}	(- 9)	23
	$+ 11011_{2c}$	+ (- 5)	+ 27
Carry out →	110010_{2c}	- 14	50

- For unsigned arithmetic, carry out indicates an overflow.
- For signed numbers, the carry out from the sign-bit is not sufficient to determine the overflow.
  - ✓It is meant for extending the adder circuit to higher order bits.
  - ✓However, overflow can be detected by observing the carry into the sign-bit position and the carry out of the sign-bit position. If these two carries are not equal, an overflow has occurred.

Overflow: 2's Complement Arithmetic for Signed Numbers

UTAR
UNIVERSITI TUNKU ABDUL RAHMAN

- The following 3 case studies are useful for detecting and analysing overflow.
- Assume B and C are positive in a 5-bit system (range represented: -16 to 15).

Case study 1: A = B + C

Compute 12 + 7.

$$\begin{array}{r} 0\ 1\ 1\ 0\ 0_{2c} \\ +\ 0\ 0\ 1\ 1\ 1_{2c} \\ \hline 1\ 0\ 0\ 1\ 1_{2c} \end{array} \qquad \begin{array}{r} 1\ 2 \\ +\ \ \ 7 \\ \hline +\ 1\ 9 \end{array}$$

Result shows an incorrect sign bit (indicates overflow)

- Addition of two positive numbers produce a negative result
- $[10011]_{2c} = -01101_2 = -13$
- -13 is incorrectly interpreted due to overflow
- Sum requires > 5 bits to represent it

Case study 2: A = - B - C

Compute (–12 – 5).

$$\begin{aligned} -12 - 5 &= (-12) + (-5) \\ &= [01100] + [00101] \\ &= 10100_{2c} + 11011_{2c} \end{aligned}$$

$$\begin{array}{r} 1\ 0\ 1\ 0\ 0_{2c} \\ +\ 1\ 1\ 0\ 1\ 1_{2c} \\ \hline 1\ 0\ 1\ 1\ 1\ 1_{2c} \end{array} \qquad \begin{array}{r} (-1\ 2) \\ +\ (-\ \ 5) \\ \hline -\ 1\ \ 7 \end{array}$$

Carry discarded

Result shows an incorrect sign bit (indicates overflow)

- Addition of two negative numbers produce a positive result
- Incorrectly interpreted as 15 due to overflow

Overflow rule for addition:
If two 2's complement numbers with the same sign are added, then overflow occurs if:
- adding two positive numbers give a NEGATIVE result.
- adding two negative numbers give a POSITIVE result.

Overflow: 2's Complement Arithmetic for Signed Numbers

- **Case study 3: A = B - C**
  - The magnitude of (B - C) will always be less than either of the two numbers.
  - Overflow can never occur

Compute 12 - 5.

12 - 5 = 12 + (-5)

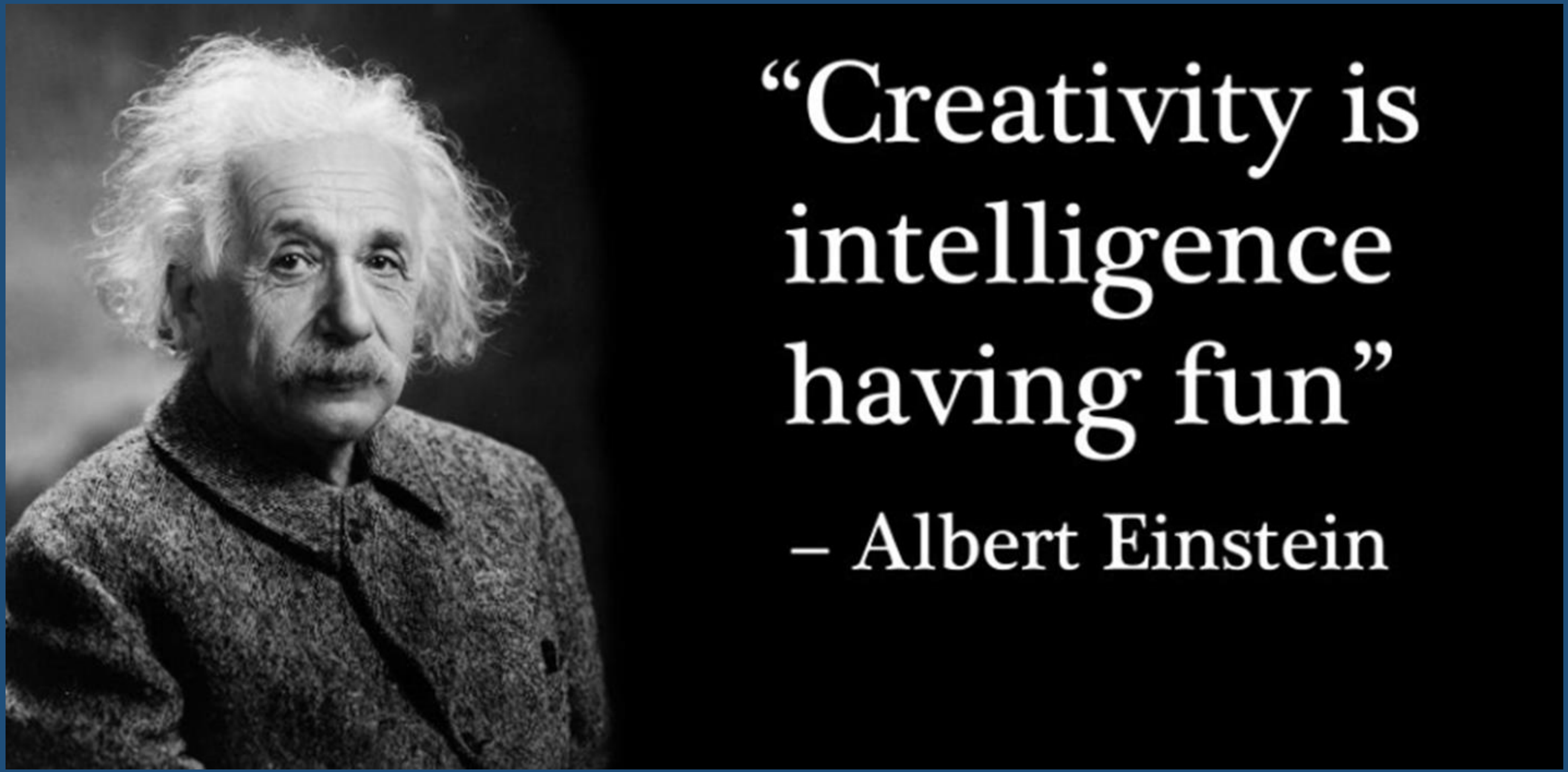
= 01100 + [00101]

= $01100_{2c} + 11011_{2c}$

$$
\begin{array}{r}
0\ 1\ 1\ 0\ 0_{2c} \\
+\ 1\ 1\ 0\ 1\ 1_{2c} \\
\hline
1\ 0\ 0\ 1\ 1\ 1_{2c}
\end{array}
\qquad
\begin{array}{r}
1\ 2 \\
+\ (-5) \\
\hline
+\ \ 7
\end{array}
$$

Carry discarded

The carry generated can be discarded
- It's not a valid carry
  - B > C shouldn't generate a carry



Q & A

Thank you